ALLIANCE UNIVERSITY · BCA (AI/ML) · DBMS ASSIGNMENT SERIES

04

ASSIGNMENT 4

Relational Database vs NoSQL Database

Online Food Delivery System

SUBMITTED BY
Vimal Kumar Rao

PROGRAMME
BCA (AI/ML)

INSTITUTION
Alliance University

SUBMISSION DATE
24 October 2026

IN THIS ASSIGNMENT

01 Compare models

02 Weigh trade-offs

03 Review cases

04 Choose the fit

A consistent relational design carried forward from Assignments 1 and 2.

1 Introduction and Brief Coverage

Relational and NoSQL databases solve overlapping but different problems. A **relational** database stores data in tables with declared relationships and constraints and is queried with SQL. **NoSQL** is a family of systems (document, key-value, wide-column, graph) that favour other data models and access patterns. NoSQL does not mean "no schema"; the schema usually lives in the application or is enforced more loosely.

	Brief requirement	Where it is covered
✓	**Compare scalability, consistency, performance, use cases**	Section 2 (comparison table) and Section 3 (ACID, BASE, CAP)
✓	**Why organisations choose different technologies**	Section 6 (decision factors) and Section 7 (weighted matrix)
✓	**Comparative report with real-world examples**	Section 5 (Ubisoft, Prime Video, Amazon, and this project)

The four NoSQL families

Family	Data shape	Typical products	Typical fit
Key-value	Opaque value looked up by key	Redis, DynamoDB	Sessions, caches, carts
Document	JSON-like documents	MongoDB, Couchbase	Catalogues, profiles, content
Wide-column	Rows with flexible column families	Cassandra, HBase	Time-series, huge write volumes
Graph	Nodes and edges	Neo4j	Social links, recommendations, fraud

2 Comparative Analysis

Dimension	Relational (RDBMS)	NoSQL	Practical implication
Data model	Tables, rows, columns; relationships through primary and foreign keys	Documents, key-value pairs, wide columns or graphs; schema flexibility varies by product	Pick the model that matches how data is created and read
Scalability	Often scales vertically first; read replicas, partitioning, sharding and distributed SQL scale further	Many products are built for horizontal partitioning across nodes	Neither is automatically "more scalable"; workload and design decide
Consistency	ACID transactions and declarative integrity rules; InnoDB supports ACID [1]	Varies by product and operation. DynamoDB reads are eventually consistent by default, with strongly consistent reads available [2]	Orders and payments need clearly defined guarantees
Querying	SQL joins, grouping, subqueries, ad-hoc analysis across tables	Queries follow keys and access patterns; multi-entity joins limited or modelled by embedding	Choose by the reads, writes and reports you need
Performance	Efficient joins, reports and transactions given good schema, indexes and queries	Very fast for well-defined, high-volume key access and denormalised reads	Benchmark real workloads; no universal winner
Schema evolution	Explicit, carefully managed ALTERs	Documents can vary and evolve; validation can still be applied [3]	Flexible fields help changing product data, validation still matters
Best-fit examples	Orders, payments, inventory, billing, bookings	Sessions, caches, telemetry, game progression, event streams	Relational core, NoSQL for suitable side workloads

3 Consistency Models: ACID, BASE and CAP

ACID (typical of relational engines)

- **A**tomicity: all of a transaction or none
- **C**onsistency: constraints always hold
- **I**solation: concurrent work does not interfere
- **D**urability: committed data survives a crash

BASE (common in distributed NoSQL)

- **B**asically **A**vailable: keeps answering during failures
- **S**oft state: replicas may briefly differ
- **E**ventual consistency: replicas converge over time

The **CAP theorem** says that when a network partition occurs, a distributed system must choose between consistency and availability. This is a statement about behaviour *during failures*, and many modern systems let you tune the trade-off per operation. For a food-delivery order, charging a customer twice or losing a paid order is far worse than a brief delay, so consistency of order and payment state comes first.

4 The Same Order in Two Models

Order 1 from this project (Asha Menon, Spice Route Kitchen, 2 x Chicken Biryani and 1 x Vegetable Meals) shown relationally and as a document.

Orders row	Order_Item rows
Order_ID 1, Customer_ID 1, Restaurant_ID 1, Address_ID 1, Delivery_ID 1, 2026-10-21 12:15:00, Delivered	(1, 1, qty 2, Rs 220.00) and (1, 3, qty 1, Rs 150.00)

```
{
  "orderId": 1, "status": "Delivered", "placedAt": "2026-10-21T12:15:00",
  "customer":   { "id": 1, "name": "Asha Menon" },
  "restaurant": { "id": 1, "name": "Spice Route Kitchen" },
  "items": [
    { "itemId": 1, "name": "Chicken Biryani", "qty": 2, "unitPrice": 220.00 },
    { "itemId": 3, "name": "Vegetable Meals", "qty": 1, "unitPrice": 150.00 }
  ],
  "payment": { "method": "UPI", "status": "Paid" }
}
```

	Relational	Document
Read one whole order	Join 3 to 5 tables	One read
Rename a restaurant	Update one row	Update every embedded copy (update anomaly)
Revenue per restaurant, per day	One GROUP BY query	Needs aggregation pipeline or a separate analytics store
Enforce "payment belongs to a real order"	Foreign key	Application code

Takeaway

The document model is excellent for reading an order as a unit, but it re-introduces the redundancy that Assignment 3 removed. This is why the transactional core of the project stays relational.

5 Real-World Examples

Relational: this project

Customers, addresses, menus, order headers, order lines and payments form a connected, transactional dataset. Keys and constraints prevent orphaned lines or payments, and questions such as "orders by restaurant", "customer spending" and "paid subtotal" need joins and aggregates, which SQL provides directly.

NoSQL: Ubisoft and DynamoDB

AWS reports that Ubisoft's Online Services team keeps gameplay-progression data for millions of players, moved its challenges database to Amazon DynamoDB from September 2022, and was handling roughly 70,000 progression data points per minute. A year later throughput was up about 43 percent with no loss of service quality, and database costs fell by about 90 percent [4]. **Why it fits:** huge write volume, simple lookups by player, variable traffic and little need for multi-table joins.

NoSQL: Prime Video and DynamoDB

Prime Video's ownership-rights service was built on Oracle in 2007. AWS describes the team migrating billions of rows to DynamoDB with no downtime, improving elasticity and reliability for a global customer base [5]. **Why it fits:** a global, always-on workload with key-based access where scale and resilience outweigh relational flexibility.

Hybrid in practice: Amazon's consumer business

When Amazon retired its last Oracle database, it moved about 75 petabytes from nearly 7,500 databases not only to DynamoDB but also to Aurora, RDS and Redshift [6]. Low-latency services went to non-relational stores; relational and analytical workloads stayed on relational engines. **Lesson:** large organisations mix technologies by workload instead of choosing one for everything.

6 Why Organisations Choose Different Technologies

Factor	Leans relational when…	Leans NoSQL when…
Data integrity	Orders, payments, stock must stay aligned	Data is disposable or rebuilt from events
Access pattern	Many different questions, ad-hoc reports	A few fixed, high-volume key lookups
Volume and traffic	Moderate, predictable load	Spiky or global load across regions
Query complexity	Joins and aggregates are routine	Reads are pre-shaped, denormalised
Schema	Stable, well-understood structure	Fields vary or change rapidly
Team and cost	SQL skills, mature tooling, low cost to start	Managed service, scale-out budget and expertise
Architecture	Single source of truth	Purpose-built side stores with a sync plan

7 Decision Matrix for This Project

Criteria are weighted by importance to the Online Food Delivery System and each option is scored from 1 (poor) to 5 (excellent). The scores are **my own illustrative judgement**, not a benchmark, and show how a requirement-driven choice is made.

Criterion	Weight	MySQL	Key-value	Document
Transactional integrity (ACID)	25	5	3	3
Relational / ad-hoc reporting	20	5	2	3
Fixed schema & clear relationships	15	5	2	3
Horizontal scale / elasticity	15	3	5	4
Team skills & tooling (coursework)	15	5	2	3

Criterion	Weight	MySQL	Key-value	Document
Flexible / evolving fields	10	3	4	5
Weighted total (out of 100)	100	90	58	67

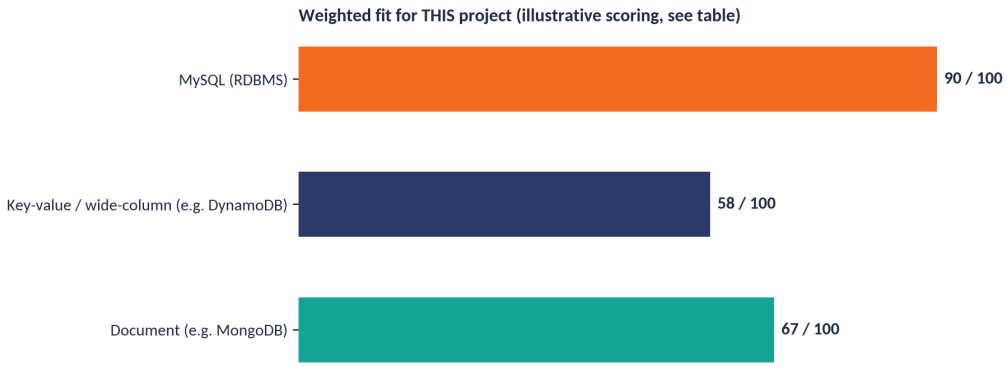


Figure 1. Weighted fit of each option for this coursework system.

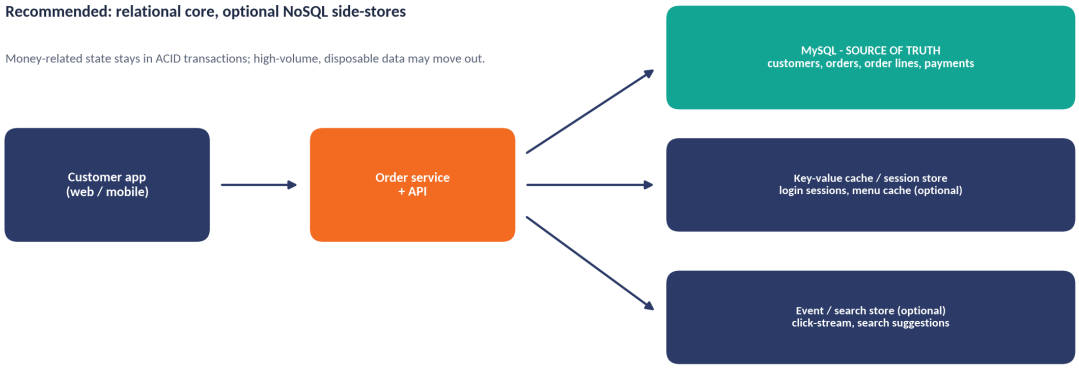


Figure 2. Recommended architecture: MySQL as the source of truth, optional NoSQL side-stores.

8 Recommendation

For the Online Food Delivery System, **MySQL is the recommended primary database**. The schema has clear entity relationships and a transactional order workflow. A future production platform could add a key-value store for sessions or menu caching and an event store for click-stream or search suggestions once measured load justifies it. Order and payment state should not move to NoSQL merely because it is popular; consistency and recovery requirements must drive the choice.

9 References

Ref	Source	URL
[1]	MySQL 8.4 Reference Manual, InnoDB and the ACID Model	https://dev.mysql.com/doc/refman/8.4/en/mysql-acid.html
[2]	Amazon DynamoDB Developer Guide, Read consistency	https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html
[3]	MongoDB Manual, Data Modeling	https://www.mongodb.com/docs/manual/data-modeling/
[4]	AWS case study, Ubisoft and Amazon DynamoDB	https://aws.amazon.com/solutions/case-studies/ubisoft-case-study/
[5]	AWS case study, Prime Video and Amazon DynamoDB	https://aws.amazon.com/solutions/case-studies/prime_video_dynamodb/
[6]	AWS News Blog, Amazon's consumer business turned off its final Oracle database	https://aws.amazon.com/blogs/aws/migration-complete-amazons-consumer-business-just-turned-off-its-final-oracle-database

Ref	Source	URL
[7]	AWS, What is NoSQL?	https://aws.amazon.com/nosql/

Accessed 24 October 2026. Vendor case studies describe results reported by the vendor. Product capabilities depend on the service and configuration.